

CAPSTONE ASSIGNMENT 1

Automate a Web Application Using Selenium WebDriver with Python

Student Name	Shreya Ghosh
Course	B.Tech — Computer Science and Business Systems (CSBS)
Year	4th Year
Enrollment No.	12023052018010
Programming Language	Python
Automation Tool	Selenium WebDriver
Test Framework	PyTest
Test Data Format	JSON
Application	Automation Exercise
Academic Year	2026

Project Report

1. Problem Statement

Automate a customer purchase workflow on a public e-commerce demo application using Selenium WebDriver with Python. The automation covers browser launch, login, product search, product selection, quantity update, cart validation, screenshot capture, external test-data reading, popup/alert handling, and execution reporting.

2. Objective

To develop a structured Selenium automation test using pytest that validates an end-to-end product purchase flow while keeping test data in a JSON file and separating reusable screenshot and popup/alert handling into utility modules.

3. Business Scenario

A customer wants to purchase a product from an e-commerce website. The automated test performs the customer journey from opening the website and logging in to searching for a product, setting its quantity, adding it to the cart, and verifying the final cart details.

4. Application and Technology

Application	Automation Exercise — public e-commerce demo application
Language	Python 3
Automation	Selenium WebDriver
Test Framework	pytest
Reporting	pytest-html
Test Data	JSON
Browser	Google Chrome

5. Tools, Software and Concepts Used

- Python 3
- Selenium WebDriver
- Google Chrome and ChromeDriver
- pytest and fixtures
- pytest-html
- JSON test data
- WebDriverWait and expected_conditions
- XPath, CSS, ID and NAME locators
- Screenshot capture
- JavaScript alert handling
- HTML modal popup handling
- Assertions

6. Project Structure

The project is divided into configuration, test, utility, report and screenshot areas so that test data and reusable functions remain separate from the main test flow.

```
02_Capstone_Project/
├── config/
│   └── test_data.json
├── tests/
│   └── test_purchase.py
├── utils/
│   ├── alerts.py
│   └── screenshots.py
├── reports/
│   └── report.html
├── screenshots/
└── requirements.txt
```

7. Test Data — JSON

The test uses JSON instead of Excel. The file stores the application URL, login credentials, product name and requested quantity.

```
{
  "url": "https://automationexercise.com/",
  "email": "YOUR_EMAIL",
  "password": "YOUR_PASSWORD",
  "product": "Blue Top",
  "quantity": 2
}
```

8. Utility Module — alerts.py

The current implementation handles a JavaScript alert when present. If no JavaScript alert exists, it waits for the HTML add-to-cart modal and clicks View Cart.

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.common.exceptions import NoAlertPresentException

def handle_javascript_alert(driver):
    try:
        alert = driver.switch_to.alert
        message = alert.text
        alert.accept()
        return message
    except NoAlertPresentException:
        return None

def handle_added_popup(driver):
    wait = WebDriverWait(driver, 10)

    popup = wait.until(
        EC.visibility_of_element_located(
            (By.XPATH, "//div[contains(@class, 'modal-content')]")
        )
    )

    popup_text = popup.text

    view_cart = wait.until(
        EC.element_to_be_clickable(
            (
                By.XPATH,
```

```

        "//div[contains(@class, 'modal-content'))"
        "//u[contains(text(), 'View Cart'))"
    )
)
)

view_cart.click()

return popup_text

```

9. Main Test — test_purchase.py

The main PyTest test loads JSON data, creates a Chrome driver through a fixture, performs the purchase flow, captures screenshots, handles the post-add-to-cart message, and validates the final cart.

```

import json
import pytest

from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

from utils.screenshots import capture_screenshot
from utils.alerts import handle_javascript_alert, handle_added_popup

with open("config/test_data.json", "r") as file:
    data = json.load(file)

@pytest.fixture
def driver():
    driver = webdriver.Chrome()
    driver.maximize_window()
    yield driver
    driver.quit()

def test_purchase_product(driver):

    wait = WebDriverWait(driver, 15)

    driver.get(data["url"])

    wait.until(
        EC.element_to_be_clickable(
            (By.XPATH, "//a[contains(text(), 'Signup / Login')]")
        )
    )

    capture_screenshot(driver, "01_home_page")

    driver.find_element(
        By.XPATH,
        "//a[contains(text(), 'Signup / Login')]"
    ).click()

    wait.until(
        EC.visibility_of_element_located(
            (By.XPATH, "//h2[contains(text(), 'Login to your account')]")
        )
    )

    driver.find_element(By.NAME, "email").send_keys(data["email"])
    driver.find_element(By.NAME, "password").send_keys(data["password"])

    driver.find_element(
        By.XPATH,
        "//button[contains(text(), 'Login')]"
    ).click()

    wait.until(
        EC.visibility_of_element_located(
            (By.XPATH, "//a[contains(text(), 'Logged in as')]")
        )
    )

```

```

capture_screenshot(driver, "02_logged_in")

driver.get(data["url"] + "view_cart")

wait.until(
    EC.visibility_of_element_located((By.ID, "cart_info"))
)

remove_buttons = driver.find_elements(
    By.CSS_SELECTOR,
    "a.cart_quantity_delete"
)

for button in remove_buttons:
    try:
        button.click()
        wait.until(EC.staleness_of(button))
    except Exception:
        pass

capture_screenshot(driver, "03_cart_cleared")

driver.find_element(
    By.XPATH,
    "//a[contains(text(),'Products')]"
).click()

wait.until(
    EC.visibility_of_element_located(
        (By.XPATH, "//h2[contains(text(),'All Products')]")
    )
)

capture_screenshot(driver, "04_products_page")

search_box = wait.until(
    EC.visibility_of_element_located((By.ID, "search_product"))
)

search_box.send_keys(data["product"])

driver.find_element(By.ID, "submit_search").click()

wait.until(
    EC.visibility_of_element_located(
        (By.XPATH, "//h2[contains(text(),'Searched Products')]")
    )
)

product = wait.until(
    EC.visibility_of_element_located(
        (
            By.XPATH,
            f"//div[contains(@class,'productinfo')]"
            f"//p[normalize-space()='{{data['product']}}']"
        )
    )
)

assert product.is_displayed()
capture_screenshot(driver, "05_product_search")

product_card = product.find_element(
    By.XPATH,
    "./ancestor::div[contains(@class,'product-image-wrapper')]"
)

view_product = product_card.find_element(
    By.XPATH,
    "//a[contains(text(),'View Product')]"
)

driver.execute_script("arguments[0].click();", view_product)

wait.until(
    EC.visibility_of_element_located((By.ID, "quantity"))
)

capture_screenshot(driver, "06_product_details")

quantity = driver.find_element(By.ID, "quantity")
quantity.clear()
quantity.send_keys(str(data["quantity"]))

assert quantity.get_attribute("value") == str(data["quantity"])

```

```

add_to_cart = wait.until(
    EC.element_to_be_clickable((By.CSS_SELECTOR, "button.cart")))
)

add_to_cart.click()

alert_text = handle_javascript_alert(driver)

if alert_text:
    print("JavaScript Alert:", alert_text)
else:
    popup_text = handle_added_popup(driver)
    print("HTML Popup:", popup_text)

capture_screenshot(driver, "07_product_added")

wait.until(
    EC.visibility_of_element_located((By.ID, "cart_info")))
)

capture_screenshot(driver, "08_cart")

cart_product = wait.until(
    EC.visibility_of_element_located(
        (
            By.XPATH,
            f"//td[contains(@class,'cart_description')]"
            f"//a[normalize-space()='{{data['product']}}']"
        )
    )
)

assert cart_product.is_displayed()

cart_quantity = wait.until(
    EC.visibility_of_element_located(
        (
            By.XPATH,
            f"//td[contains(@class,'cart_description')]"
            f"//a[normalize-space()='{{data['product']}}']"
            f"/ancestor::tr"
            f"//td[contains(@class,'cart_quantity')]/button"
        )
    )
)

assert cart_quantity.text == str(data["quantity"])

capture_screenshot(driver, "09_final_cart")

```

10. Automation Workflow

1. Launch Chrome and open Automation Exercise.
2. Open Signup / Login and enter credentials from JSON.
3. Verify the logged-in state.
4. Open the cart and remove existing items so the test starts clean.
5. Navigate to Products and search for the configured product.
6. Verify that the searched product is displayed.
7. Open product details and set the requested quantity.
8. Add the product to the cart.
9. Check for a JavaScript alert; otherwise handle the HTML add-to-cart popup.
10. Verify the product and quantity in the final cart.
11. Capture screenshots throughout execution.

11. Output

The test captures screenshots at the defined checkpoints and prints either the JavaScript alert text or the HTML popup text after the product is added. The PyTest HTML execution report is generated in the reports folder.

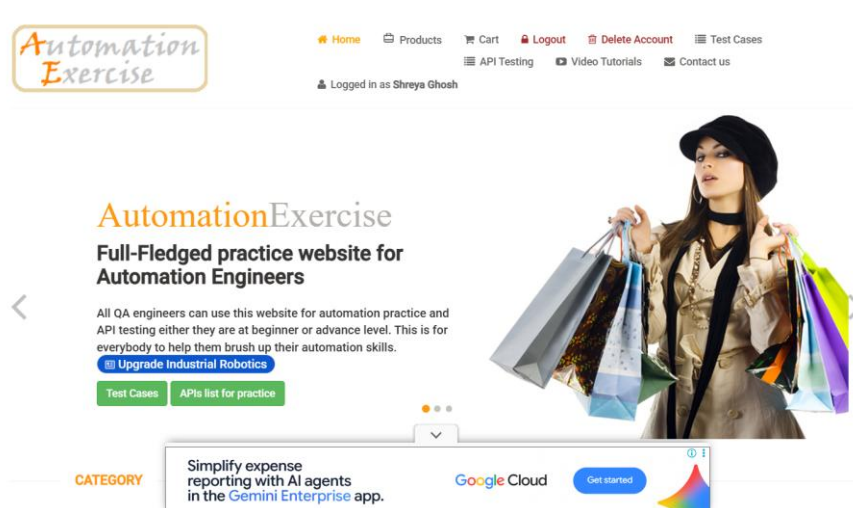


Figure 1. Home Page

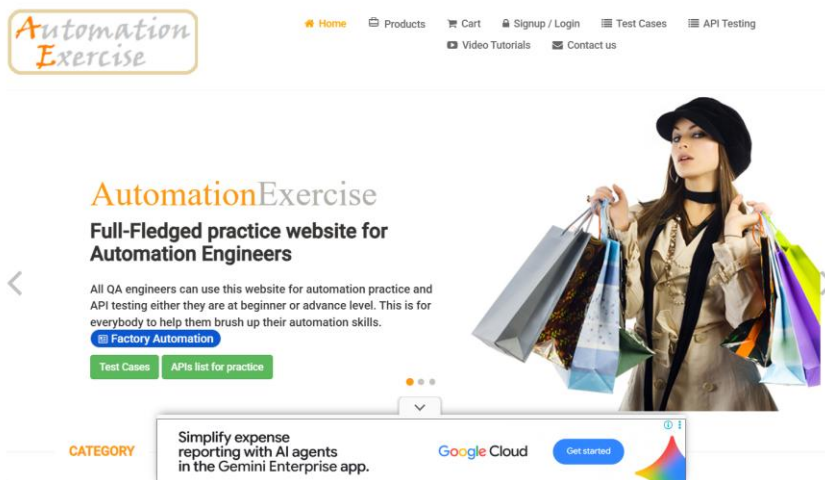


Figure 2.
Successful Login

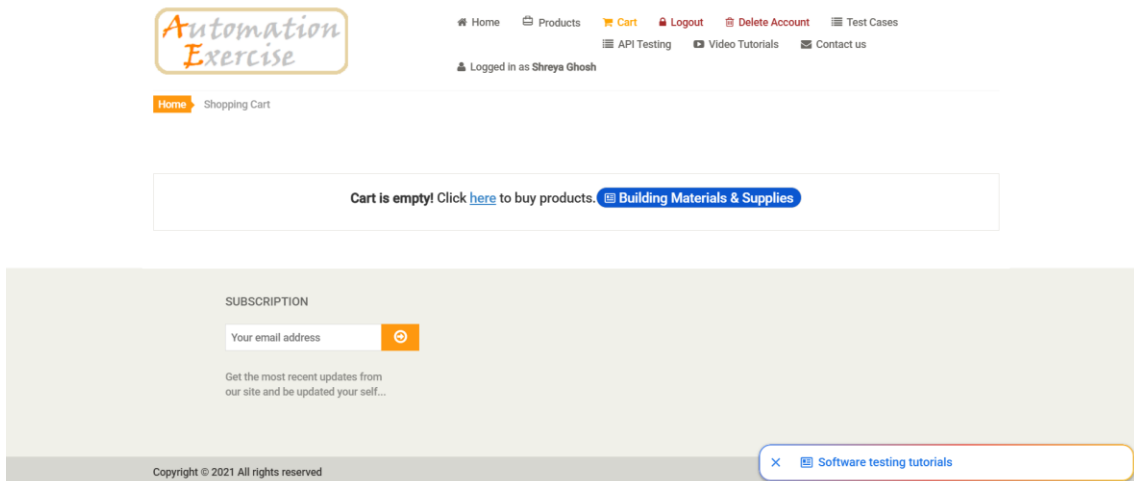


Figure 3. Cart
Cleared

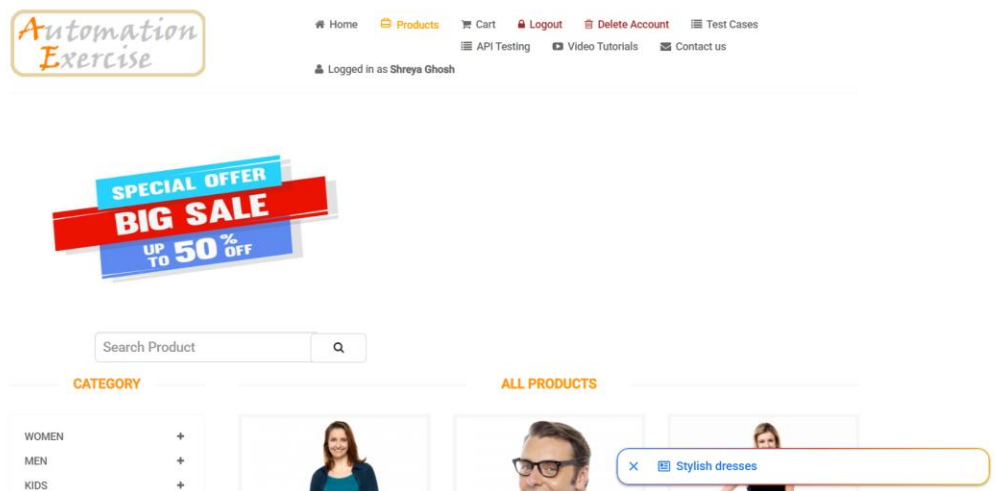


Figure 4.
Products Page

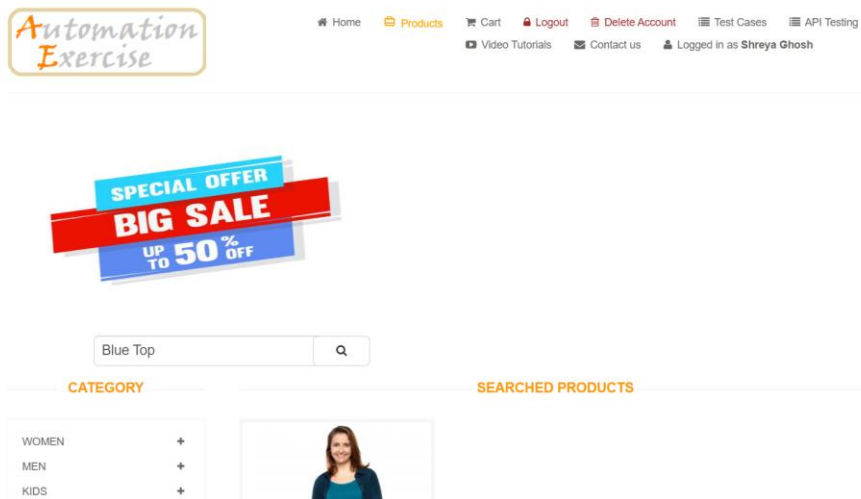


Figure 5. Product Search Result

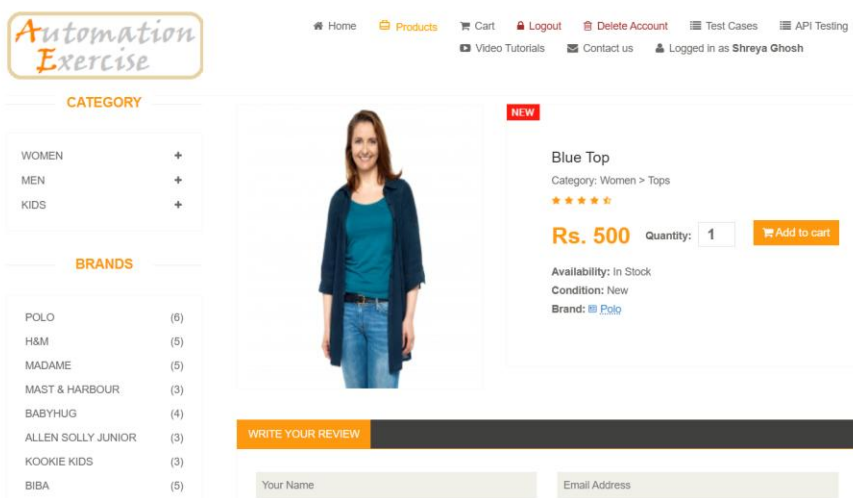


Figure 6. Product Details and Quantity

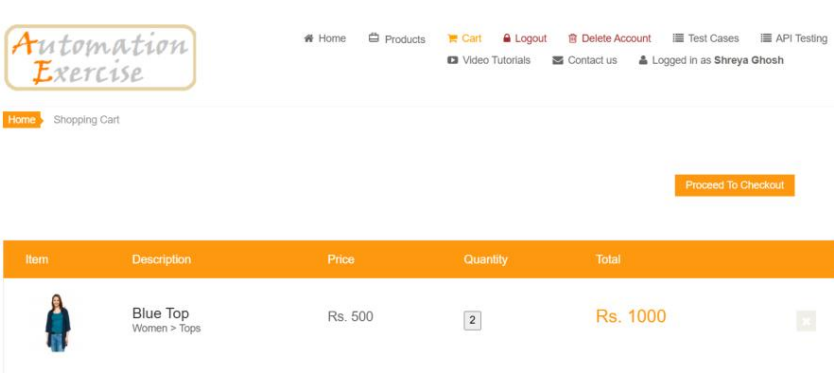


Figure 7. Product Added to Cart

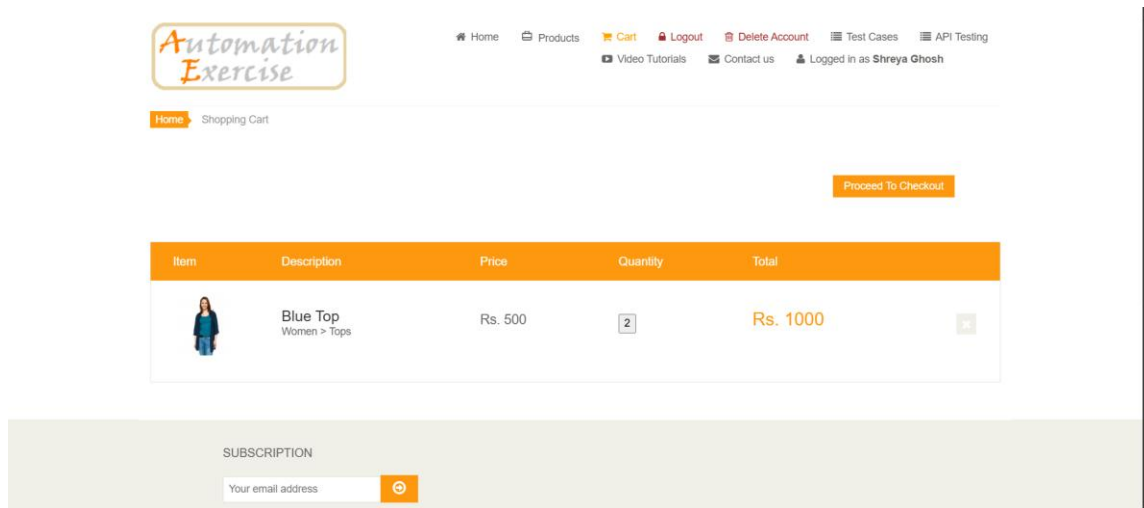


Figure 8. Cart Details

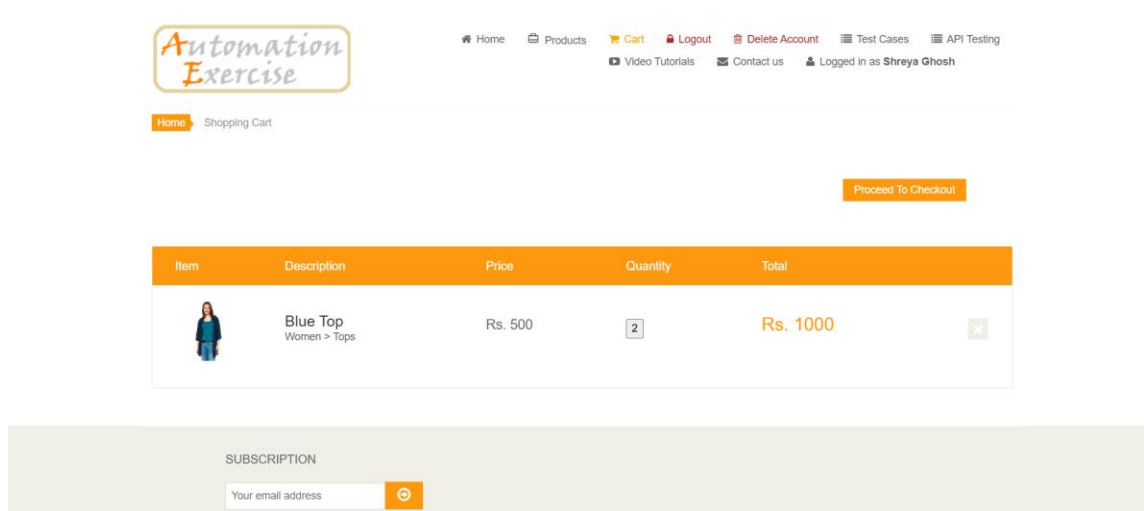


Figure 9. Final Cart Validation

13. Execution Report

The project uses pytest-html to generate a self-contained HTML execution report.

```
python -m pytest tests/test_purchase.py --html=reports/report.html --self-contained-html
```

The generated report is saved as reports/report.html.

14. Result

The implemented automation covers the intended e-commerce workflow: login, product search, product selection, quantity entry, cart preparation, add-to-cart popup handling, and final cart validation.

15. Conclusion

The experiment demonstrated an end-to-end Selenium WebDriver automation project using Python and PyTest. The implementation combines structured test organization, JSON-based test data, explicit waits, assertions, screenshots, popup/alert handling and HTML execution reporting.